

Winscorin Breast Cancer Detection Report

Introduction

Breast cancer happens because damaged cells in the breast area keep multiplying and do not self-destruct. Early detection helps to localize these cancerous cells, which are easier to treat and manage. By managing the cancerous cells locally, patients not only avoid harsh chemotherapy treatment but also lower the risk of the cancerous cells resurfacing, even after medical treatments. The five-year survival rate for breast cancer approaches 100% when detected at its earliest stage, falling to as low as 15% when detected at the latest stage (Yue et al., 2018), underscoring the critical importance of early and accurate diagnosis. Traditionally, diagnostic accuracy depends on physician experience, which can be overwhelming and cannot be guaranteed (Yue et al., 2018). Machine Learning offers a more consistent and measurable approach. The Wisconsin breast cancer dataset, publicly available through the UCI machine learning repository, was used. The K-Nearest Neighbours (KNN) and Decision Tree (DT) algorithms were used. The dataset has a total of 699 patients, 9 features, binary target (benign/malignant).

```
> str(data)
'data.frame': 699 obs. of 10 variables:
 $ Cl.thickness : int  5 5 3 6 4 8 1 2 2 4 ...
 $ Cell.size    : int  1 4 1 8 1 10 1 1 1 2 ...
 $ Cell.shape   : int  1 4 1 8 1 10 1 2 1 1 ...
 $ Marg.adhesion : int  1 5 1 1 3 8 1 1 1 1 ...
 $ Epith.c.size : int  2 7 2 3 2 7 2 2 2 2 ...
 $ Bare.nuclei  : int  1 10 2 4 1 10 10 1 1 1 ...
 $ Bl.cromatin   : int  3 3 3 3 3 9 3 3 1 2 ...
 $ Normal.nucleoli: int  1 2 1 7 1 7 1 1 1 1 ...
 $ Mitoses       : int  1 1 1 1 1 1 1 1 5 1 ...
 $ Class         : chr  "benign" "benign" "benign" "benign" ...
```

Figure 1

The columns were all integers except the binary-target label. Each cell represents cell measurements scored 1-10 by doctors- the higher the number, the higher the chances of the decision being malignant- across all features. The dataset has a class imbalance representing 458 benign and 241 malignant cases.



Figure 2

This dataset is suitable for ML classification because its target label is binary.

The DT algorithm was used because it aids interpretability to doctors on how the decision was arrived at, also placing importance on which features have more weight of importance. The KNN bases its result on the feature similarity of the closest neighbours by classifying new patients based on their closest neighbours' similar feature values, which suggests a similar diagnosis. These 2 algorithms are perfect for this dataset because they show side-by-side the performance of the black-box algorithm (KNN) and the explainable model (DT). Furthermore, both are supervised classification algorithms, good for binary targets.

Pre-processing

Data Exploration

The data was loaded into RStudio and explored. I confirmed the file was within the working directory using the `getwd()` and `list.files()`. I explored using `summary(data)`, `dim(data)`, `str(data)`, and `ColSums(is.na(data))` to inspect the data, its dimensions, types, and missing values.

```
> summary(data)
Cl.thickness      Cell.size      Cell.shape      Marg.adhesion    Epith.c.size
Min.   : 1.000    Min.   : 1.000    Min.   : 1.000    Min.   : 1.000    Min.   : 1.000
1st Qu.: 2.000    1st Qu.: 1.000    1st Qu.: 1.000    1st Qu.: 1.000    1st Qu.: 2.000
Median : 4.000    Median : 1.000    Median : 1.000    Median : 1.000    Median : 2.000
Mean   : 4.418    Mean   : 3.134    Mean   : 3.207    Mean   : 2.807    Mean   : 3.216
3rd Qu.: 6.000    3rd Qu.: 5.000    3rd Qu.: 5.000    3rd Qu.: 4.000    3rd Qu.: 4.000
Max.   :10.000    Max.   :10.000    Max.   :10.000    Max.   :10.000    Max.   :10.000

Bare.nuclei      Bl.cromatin      Normal.nucleoli  Mitoses          Class
Min.   : 1.000    Min.   : 1.000    Min.   : 1.000    Min.   : 1.000    Length :699
1st Qu.: 1.000    1st Qu.: 2.000    1st Qu.: 1.000    1st Qu.: 1.000    N.unique : 2
Median : 1.000    Median : 3.000    Median : 1.000    Median : 1.000    N.blank  : 0
Mean   : 3.545    Mean   : 3.438    Mean   : 2.867    Mean   : 1.589    Min.nchar: 6
3rd Qu.: 6.000    3rd Qu.: 5.000    3rd Qu.: 4.000    3rd Qu.: 1.000    Max.nchar: 9
Max.   :10.000    Max.   :10.000    Max.   :10.000    Max.   :10.000
NAS      :16
```

Figure 3

Missing Value

As shown in Figure 4 above, there were a total of 16 missing values from the **Bare. Nuclei** column.

Clinically, the range of 1-10 of each feature depicts the gravity of each patient's situation towards malignant (Yang, Zhang, and Liu, 2018). The higher the aggregated value of each feature, the more likely the tumor is malignant. Replacing the missing values rather than

deleting rows preserves more data for the model to learn from. In replacing, either the mean or median value could work depending on what I'm optimizing the model for.

The model will aim to minimize false negative predictions, failure of which may lead to untreated breast cancer and death. From the `summary(data$Bare.nuclei)` below, the mean value of 3.545 is higher than the median value of 1, which is positively skewed. Using the mean value, which is high, signals the model's sensitivity to the higher end of each range, favoring the model's ability to be sensitive to malignant cases. The distribution is bimodal: 402 patients score 1 and 132 score 10. Using the mean (~4), the count at 4 rises from 19 to 35 after imputation.

```
> table(data$Bare.nuclei)

 1    2    3    4    5    6    7    8    9   10 
402   30   28   35   30    4    8   21    9  132
```

Figure 4

Normalisation decision

Normalization was unnecessary, as all features share the same scale from 1- 10 (figure 3)

Class imbalance and split

Using the library package, GGplot, the class imbalance was visualized, with the data points having a biased distribution of benign cases having almost twice the number of malignant cases. Already, this imbalance poses a compromise to the model's best performance on learning as much as it can on malignant cases, enough to be both confident in predicting on unseen cases and also to have a high sensitivity score. However, using synthetic data to generate more malignant datapoints to either match or, better still, surpass the number of benign cases will be a great option to maximize the model's performance.

The data was split into training and test data, with a random seed of 42 set to ensure consistent selection of the training data for each run.

Model Development

KNN Baseline Model

For the KNN model, the features were separated from the target label into three objects: `train_features` (the 489 training rows with 9 columns), `test_features` (201 test rows with 9 columns), and `train_labels` (the benign/malignant classes). KNN needs these separately because the algorithm measures distance between feature values to find a new patient's

nearest neighbours; if the class label were left inside the feature set, the model would effectively be answered and could not learn to predict it.

K determines how many neighboring patients the model consults before casting a majority vote. A K that is too small (e.g., K= 1) makes the model sensitive to noise- a single mislabelled or unusual neighbour can swing the prediction, causing overfitting. A K that is too large smooths over the genuine local patterns and lets the majority class (benign) dominate, causing underfitting and a drop in sensitivity. The square root of the training sample size offers a balanced midpoint between these extremes (Amir et al., 2019), which is why K= 21 was chosen as the baseline. The baseline result was the weakest of all values tested: 8 false negatives, sensitivity 87.5%, accuracy 93.8%, specificity 96.6%, clearly leaving room to improve sensitivity, the metric that matters in this clinical context.

KNN Hyper-tuning

To reduce false negatives, smaller K values were tested (1, 2, 3, 5). Smaller K lets the model respond to local patterns rather than being diluted by the larger benign majority, which generally lifts sensitivity. The results confirmed this; every smaller value outperformed K=21. K=2 emerged as the best, with sensitivity 93.8% and accuracy 95.7%. Although K=3 matched that accuracy (95.7%), K=2 was selected because it achieved the higher sensitivity- the deciding factor in a clinical screening where missing a malignant case is the most costly error.

```
print(knn_summary)
```

K	TP	TN	FP	FN	Accuracy	Sensitivity	Specificity
21	56	141	5	8	0.9381	0.8750	0.9658
5	59	141	5	5	0.9524	0.9219	0.9658
3	59	142	4	5	0.9571	0.9219	0.9726
1	59	141	5	5	0.9524	0.9219	0.9658
2	60	141	5	4	0.9571	0.9375	0.9658

Figure 5

Decision Tree

The formula `rpart(Class ~ ., data = train_data)` instructs the model to predict the Class label using all other columns as predictors (the `'.'` means 'every remaining feature'). The tree's first split was on `Cell.size`, meaning the model identified uniformity of cell size as the single most informative feature for separating benign from malignant cases. This is clinically meaningful, as enlarged and irregular cell size is a recognized hallmark of malignancy. On the test set, the DT

achieved an accuracy of 93.3%, a sensitivity of 89.1%, and a specificity of 95.2%, which were lower than those of KNN (K=2) on all three metrics, but still strong.

Custom Function

R's built-in tools report accuracy easily, but accuracy alone is misleading on an imbalanced dataset- a model could score well simply by favouring the majority benign class while missing malignant cases. A custom *calculate_metrics* function was written to return accuracy, sensitivity, and specificity from the confusion matrix, ensuring the clinically critical metrics were visible and appropriately tested. Sensitivity reveals how many true malignant cases were caught (minimizing deadly false negatives); specificity reveals how many benign cases were correctly cleared (minimizing unnecessary anxiety and biopsies).

```
calculate_metrics <- function(true_positive, true_negative,
                             false_positive, false_negative)
{
  accuracy <- (true_positive + true_negative) / (true_negative + true_positive
                                                + false_negative + false_positive)
  sensitivity <- true_positive / (true_positive + false_negative)
  specificity <- true_negative / (true_negative + false_positive)
  return(c(Accuracy = accuracy, Sensitivity = sensitivity, Specificity = specificity))
}

calculate_metrics(true_positive, true_negative,
                 false_positive, false_negative)
```

Figure 6

Model Evaluation and Recommendation

Class Distribution

Figure 2 shows 458 benign and 241 malignant cases; benign cases are almost twice as common. This imbalance means the model sees far fewer malignant datapoints for it to learn more from, exposing it to bias towards predicting benign and risking lower sensitivity. A technique such as SMOTE, which generates synthetic malignant cases to rebalance the classes, could address this. It was noted as fixture work rather than applied here to keep the baseline analysis transparent and reproducible.

KNN Performance Across K Values

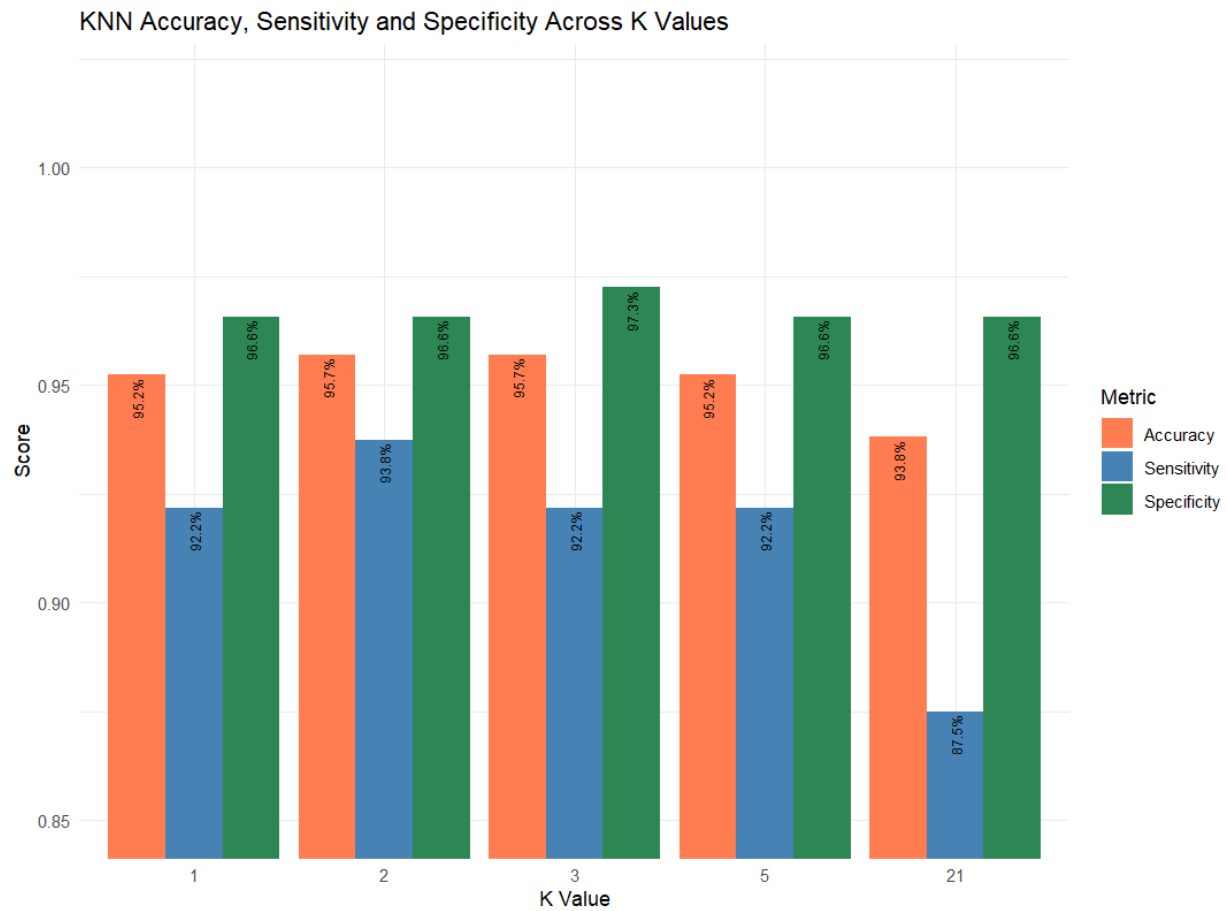


Figure 7

As K increases, sensitivity declines, peaking at K=2 (93.8%) and falling to its lowest at K=21. This makes clinical sense: smaller K lets the model respond to the local patterns of individual malignant cases, whereas larger K averages predictions across more neighbours and lets the benign majority dominate.

Model Comparison

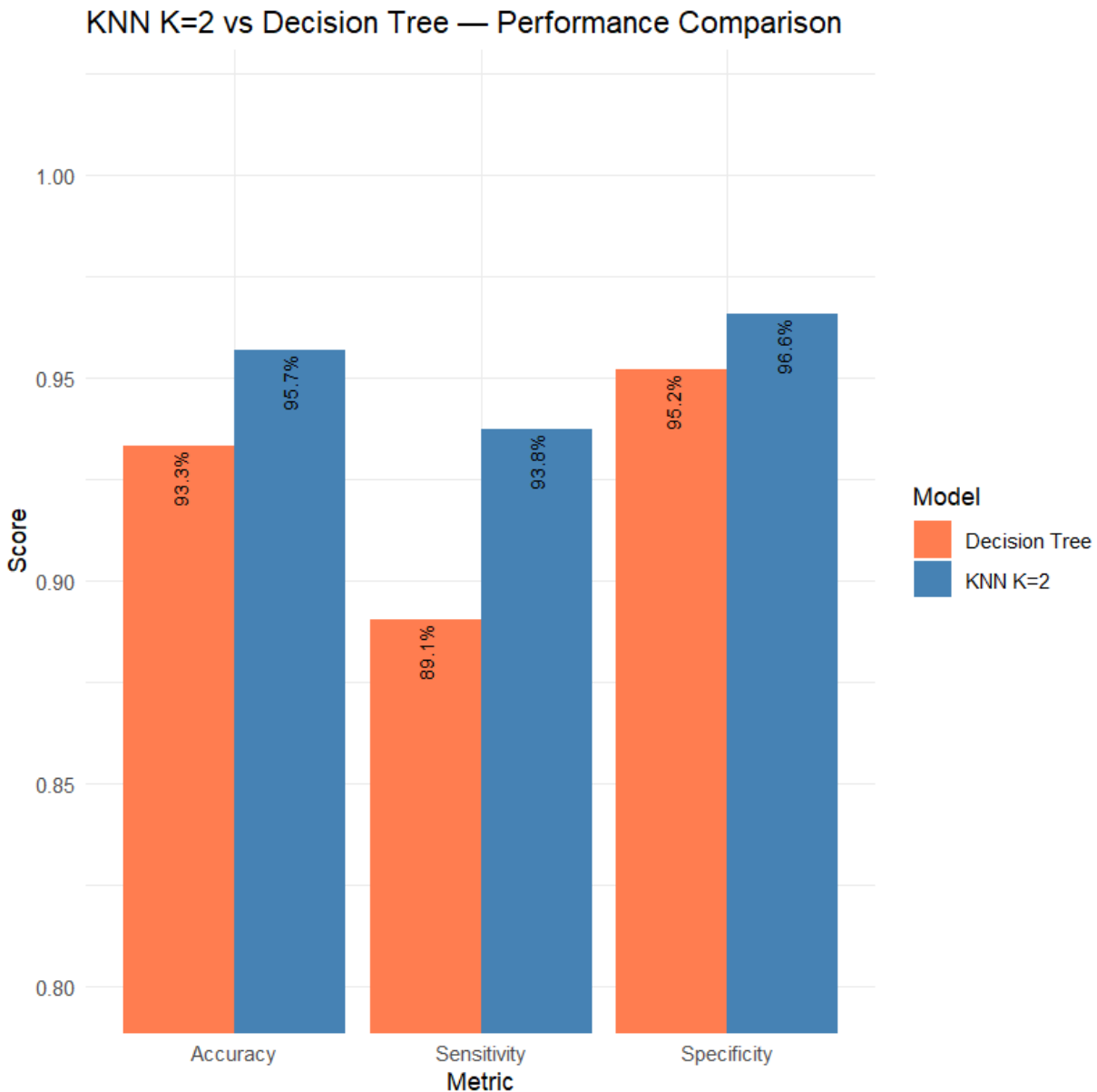


Figure 8

KNN K=2 leads on all three metrics: accuracy (95.7% vs 93.3%), sensitivity (93.8% vs 89.1%), and specificity (96.6% vs 95.2%). The DT comes closest to specificity. Both models classified a similar number of cases correctly, suggesting the two groups are fairly easy to tell apart. Malignant cases tend to score high on the 1-10 scale and benign cases low, with little overlap, so even a simple model can separate them well.

Clinical Recommendation

KNN K=2 is recommended for deployment, given its superior sensitivity (93.8%)- the priority metric for screening. The DT's advantage is its traceable, auditable decision path, which KNN's black-box approach cannot offer.

This report set out to build and to compare machine learning models for classifying breast tumors as benign or malignant using the Wisconsin Breast Cancer Dataset and successfully delivered two working classifiers with strong predictive performance. The recommended model is KNN with K=2, achieving accuracy of 95.7%, sensitivity of 93.8%, and specificity of 96.6%. Although the DT scored lower across all metrics, it brings essential explainability to clinical practice, offering a transparent decision path that supports clinician trust and accountability.

Two limitations should be noted: the dataset's class imbalance (458 benign vs 241 malignant) may bias the model towards the majority class, and the analysis relied on a single dataset, limiting how far the results generalize to other populations. Future work would address these in three steps: applying SMOTE to rebalance the classes and strengthen malignant-case learning; testing a pruned DT or ensemble method to lift its metrics while keeping interpretability; and validating the pipeline on a larger, real-world clinical dataset ahead of any NHS deployment.

A third caveat is that K=2 relies on only two neighbours, so its strong result may partly reflect the single train/test split; testing across multiple splits would confirm readability.

References

1. Yang, J., Zhang, P. and Liu, Y. (2018) 'Robustness of classification ability of spiking neural networks', *arXiv preprint*, arXiv:1801.09827. Available at: <https://arxiv.org/pdf/1801.09827>
2. Amir, S. et al. (2019) 'Supervised Machine Learning Techniques for Trojan Detection with Ring Oscillator Network', *arXiv preprint*, arXiv:1903.04677. Available at: <https://arxiv.org/pdf/1903.04677>

Source Code

R packages used: ggplot2, class, rpart, tidyr. All code is my own.

```

# Source Code Listing – Wisconsin Breast Cancer Prediction

# This is a supervised ML task: the target label Class is either
# 'benign' or 'malignant', which makes this a binary classification problem.
# Two classifiers are applied and compared: KNN and Decision Tree.

# ---- LOAD THE DATA ----
getwd()                # confirm working directory
list.files()           # confirm the dataset is in the folder
data <- read.csv("winsconsin.csv")

# ---- PREVIEW THE DATA ----
summary(data)          # statistical snapshot of the data
dim(data)              # number of rows and columns
str(data)              # data type of each feature
sum(is.na(data))       # total missing values

# 16 missing values found; identify which column(s)
colSums(is.na(data))   # missing values are in the Bare.nuclei column

# Inspect the Bare.nuclei column
summary(data$Bare.nuclei)
table(data$Bare.nuclei) # frequency of each value (mode check)

# Bimodal distribution: 402 patients score 1, 132 score 10.
# Higher feature values clinically indicate greater likelihood of malignancy.
# Missing values are imputed (not deleted) to preserve data for the model.
# The mean (3.545) is chosen over the median (1): a higher fill value tilts
# the model toward sensitivity to malignant cases, which is the priority,
# since false negatives (missed cancers) are the most costly error.

# Replace missing values in Bare.nuclei with the rounded mean
data$Bare.nuclei[is.na(data$Bare.nuclei)] <- round(mean(data$Bare.nuclei, na.rm
= TRUE), 0)
table(data$Bare.nuclei)
sum(is.na(data))       # verify no missing values remain

# ---- NORMALISATION ----
# All features share a uniform 1-10 scale, so normalisation is not required.
summary(data)

# ---- FACTORISE THE TARGET LABEL ----
data$Class <- as.factor(data$Class)
table(data$Class)      # 458 benign, 241 malignant (class imbalance)

```

```

summary(data$Class)

# ---- CLASS DISTRIBUTION PLOT ----
install.packages("ggplot2")
library(ggplot2)

ggplot(data, aes(x = Class, fill = Class)) +
  geom_bar() +
  geom_text(stat = "count", aes(label = after_stat(count)),
            vjust = -0.5, size = 4) +
  labs(title = "Class Distribution - Benign vs Malignant",
        x = "Tumour Class",
        y = "Number of Patients") +
  theme_minimal()

# The data is imbalanced (benign nearly double malignant). SMOTE could
# generate synthetic malignant cases to rebalance; noted as future work.

# ---- SPLIT THE DATA (70/30) ----
set.seed(42) # reproducible split
train_index <- sample(1:nrow(data), nrow(data) * 0.70, replace = FALSE)
train_data <- data[train_index, ]
test_data <- data[-train_index, ]

dim(train_data) # 489 rows
dim(test_data) # 210 rows

# ---- PREPARE FEATURES AND LABELS FOR KNN ----
install.packages("class")
library(class)

train_features <- train_data[, -which(names(train_data) == "Class")]
test_features <- test_data[, -which(names(test_data) == "Class")]
train_labels <- train_data$Class

str(train_features)
str(test_features)
str(train_labels)

# ---- KNN BASELINE (K = 21, sqrt of 489) ----
knn_predictions <- knn(train = train_features, test = test_features,
                       cl = train_labels, k = 21)

conf_matrix <- table(knn_predictions, test_data$Class)
conf_matrix

```

```

# Malignant = positive, benign = negative.
# Rows are predicted outcomes, columns are actual outcomes.
true_negative <- conf_matrix[1, 1]
false_positive <- conf_matrix[2, 1]
false_negative <- conf_matrix[1, 2]
true_positive <- conf_matrix[2, 2]

# ---- CUSTOM METRICS FUNCTION ----
# Accuracy alone is misleading on imbalanced data, so this returns
# accuracy, sensitivity and specificity together.
calculate_metrics <- function(true_positive, true_negative,
                              false_positive, false_negative) {
  accuracy <- (true_positive + true_negative) /
    (true_negative + true_positive + false_negative +
false_positive)
  sensitivity <- true_positive / (true_positive + false_negative)
  specificity <- true_negative / (true_negative + false_positive)
  return(c(Accuracy = accuracy, Sensitivity = sensitivity, Specificity =
specificity))
}

calculate_metrics(true_positive, true_negative, false_positive, false_negative)

# ---- HYPER-TUNING: K = 5 ----
knn_predcitions_1sthypertune <- knn(train = train_features, test =
test_features,
                                     cl = train_labels, k = 5)
conf_matrix_1sthypertune <- table(knn_predcitions_1sthypertune,
test_data$Class)
conf_matrix_1sthypertune

true_negative_1h <- conf_matrix_1sthypertune[1, 1]
false_positive_1h <- conf_matrix_1sthypertune[2, 1]
false_negative_1h <- conf_matrix_1sthypertune[1, 2]
true_positive_1h <- conf_matrix_1sthypertune[2, 2]

# ---- HYPER-TUNING: K = 3 ----
knn_predcitions_2ndhypertune <- knn(train = train_features, test =
test_features,
                                     cl = train_labels, k = 3)
conf_matrix_2ndhypertune <- table(knn_predcitions_2ndhypertune,
test_data$Class)
conf_matrix_2ndhypertune

true_negative_2h <- conf_matrix_2ndhypertune[1, 1]

```

```

false_positive_2h <- conf_matrix_2ndhypertune[2, 1]
false_negative_2h <- conf_matrix_2ndhypertune[1, 2]
true_positive_2h <- conf_matrix_2ndhypertune[2, 2]

# ---- HYPER-TUNING: K = 1 ----
knn_predcitions_3rdhypertune <- knn(train = train_features, test =
test_features,
                                cl = train_labels, k = 1)
conf_matrix_3rdhypertune <- table(knn_predcitions_3rdhypertune,
test_data$Class)
conf_matrix_3rdhypertune

true_negative_3h <- conf_matrix_3rdhypertune[1, 1]
false_positive_3h <- conf_matrix_3rdhypertune[2, 1]
false_negative_3h <- conf_matrix_3rdhypertune[1, 2]
true_positive_3h <- conf_matrix_3rdhypertune[2, 2]

# ---- HYPER-TUNING: K = 2 (best model) ----
knn_predcitions_4thhypertune <- knn(train = train_features, test =
test_features,
                                cl = train_labels, k = 2)
conf_matrix_4thhypertune <- table(knn_predcitions_4thhypertune,
test_data$Class)
conf_matrix_4thhypertune

true_negative_4h <- conf_matrix_4thhypertune[1, 1]
false_positive_4h <- conf_matrix_4thhypertune[2, 1]
false_negative_4h <- conf_matrix_4thhypertune[1, 2]
true_positive_4h <- conf_matrix_4thhypertune[2, 2]

# ---- DECISION TREE ----
# Explainability matters in medicine: a Decision Tree is a flowchart of
# yes/no splits that lets clinicians follow how a prediction was reached,
# and it weighs the most important features. It needs no feature scaling.
install.packages("rpart")
library(rpart)

dt_model <- rpart(Class ~ ., data = train_data) # predict Class from all
features

par(mar = c(1, 1, 1, 1)) # widen plot margins
plot(dt_model, uniform = TRUE, margin = 0.1)
text(dt_model, use.n = TRUE, cex = 0.7)

dt_predictions <- predict(dt_model, newdata = test_data, type = "class")
conf_matrix_dt <- table(dt_predictions, test_data$Class)

```

```

conf_matrix_dt

true_negative_dt <- conf_matrix_dt[1, 1]
false_positive_dt <- conf_matrix_dt[2, 1]
false_negative_dt <- conf_matrix_dt[1, 2]
true_positive_dt <- conf_matrix_dt[2, 2]

calculate_metrics(true_positive_dt, true_negative_dt, false_positive_dt,
false_negative_dt)

# ---- COMPARE ALL KNN MODELS ----
calculate_metrics(true_positive, true_negative, false_positive,
false_negative) # K=21
calculate_metrics(true_positive_1h, true_negative_1h, false_positive_1h,
false_negative_1h) # K=5
calculate_metrics(true_positive_2h, true_negative_2h, false_positive_2h,
false_negative_2h) # K=3
calculate_metrics(true_positive_3h, true_negative_3h, false_positive_3h,
false_negative_3h) # K=1
calculate_metrics(true_positive_4h, true_negative_4h, false_positive_4h,
false_negative_4h) # K=2

# ---- KNN METRICS ACROSS K VALUES ----
library(tidyr)

knn_results <- data.frame(
  K = c(1, 2, 3, 5, 21),
  Accuracy = c(0.9524, 0.9571, 0.9571, 0.9524, 0.9381),
  Sensitivity = c(0.9219, 0.9375, 0.9219, 0.9219, 0.8750),
  Specificity = c(0.9658, 0.9658, 0.9726, 0.9658, 0.9658)
)

knn_long <- pivot_longer(knn_results,
  cols = c(Accuracy, Sensitivity, Specificity),
  names_to = "Metric", values_to = "Value")

ggplot(knn_long, aes(x = factor(K), y = Value, fill = Metric)) +
  geom_bar(stat = "identity", position = "dodge") +
  geom_text(aes(label = paste0(round(Value * 100, 1), "%")),
    position = position_dodge(width = 0.9),
    vjust = 0.5, hjust = 1.1, angle = 90, size = 2.5) +
  labs(title = "KNN Accuracy, Sensitivity and Specificity Across K Values",
    x = "K Value", y = "Score") +
  coord_cartesian(ylim = c(0.85, 1.02)) +
  scale_fill_manual(values = c("Accuracy" = "coral",
    "Sensitivity" = "steelblue",
    "Specificity" = "seagreen")) +

```

```

theme_minimal()

# ---- KNN K=2 vs DECISION TREE ----
model_comparison <- data.frame(
  Model = c("KNN K=2", "Decision Tree"),
  Accuracy = c(0.9571, 0.9333),
  Sensitivity = c(0.9375, 0.8906),
  Specificity = c(0.9658, 0.9521)
)

model_long <- pivot_longer(model_comparison,
                           cols = c(Accuracy, Sensitivity, Specificity),
                           names_to = "Metric", values_to = "Value")

ggplot(model_long, aes(x = Metric, y = Value, fill = Model)) +
  geom_bar(stat = "identity", position = "dodge") +
  geom_text(aes(label = paste0(round(Value * 100, 1), "%")),
            position = position_dodge(width = 0.9),
            vjust = 0.5, hjust = 1.1, angle = 90, size = 3) +
  labs(title = "KNN K=2 vs Decision Tree - Performance Comparison",
       x = "Metric", y = "Score") +
  coord_cartesian(ylim = c(0.80, 1.02)) +
  scale_fill_manual(values = c("KNN K=2" = "steelblue",
                              "Decision Tree" = "coral")) +
  theme_minimal()

```